Savneet Frontend Guide

Simple explanation of how the ALI website works, what files are used, what dependencies are installed, and what each frontend part is responsible for.

Tiny version first

The frontend is the pretty control panel for ALI. Users see it, click it, upload files, press Run ALI, and watch the bridge-building process happen.

Frontend = the face

It shows the landing page, signup/login screens, upload area, bridge timeline, generated Python code, and final result.

Backend = the engine

The frontend sends a request to the Flask backend at http://localhost:5000/run. The backend runs the sandbox and sends the result back.

ALI = the smart bridge builder

ALI reads API docs, understands fields, writes bridge code, runs it safely, and reports success or failure.

How the app flows

User journey

- User opens the website at http://localhost:5173.
- Landing page explains the idea: connect any two tools with no code.
- User can go to signup/login, then enter the ALI tool interface.
- Inside the tool page, user uploads two API documentation files.
- User writes an instruction like: when a HubSpot lead arrives, send it to Discord.
- User clicks Run ALI.
- Frontend calls the backend /run API.
- Frontend shows progress, generated Python code, and final result.

Explain it to judges

The website is not just decoration. It is the user-facing interface that demonstrates the full ALI pipeline from documents to generated bridge code to sandbox execution.

Main files

Important frontend files

frontend/src/main.jsx

Starts the React app and places it inside the browser page.

frontend/src/App.jsx

Controls which page is visible: landing, signup, login, or tool page.

frontend/src/index.css

All major styling: pastel cream background, maroon text, glassmorphism, responsive layout, buttons, cards, and animations.

frontend/src/api/aliClient.js

Talks to the backend /run route and normalizes the response so the UI can display it.

frontend/src/data/pipeline.js

Defines the steps shown in the progress timeline.

Components

What each component does

Navigation.jsx

Top navbar. Shows the ALI logo, See how it works, Log In, and Sign Up buttons.

LandingPage.jsx

The main homepage with hero text, 3D-style orbit artwork, services section, team section, demo preview, and CTA.

AuthPage.jsx

Login and signup screen styling. It is visual/demo focused right now, not a real auth system.

ToolPage.jsx

Main working interface where users load docs, write the task, run ALI, and view results.

UploadZone.jsx

Reusable upload boxes for Tool A and Tool B docs.

More components

Visual and result components

NetworkCanvas.jsx

Animated background network dots and lines. This gives the site a living technology feel.

PipelineTimeline.jsx

Shows ALI's steps: reading docs, extracting fields, mapping, writing code, and running.

CodeWindow.jsx

Displays the generated Python bridge code in a styled code window.

ResultCard.jsx

Shows whether ALI succeeded, failed, used fallback mode, and how many attempts happened.

assets/ali-logo-mark.png

Transparent logo mark used in the navbar.

Dependencies

Main packages used

React
Builds the UI using reusable components.

React DOM
Places the React app into the browser's HTML page.

Vite
Runs the local frontend server and builds the final production files.

GSAP
Creates smooth animations, page transitions, scroll animations, and entrance effects.

Three
Available for 3D-style visuals and advanced interactive graphics.

PrismJS
Used for nice code highlighting in generated code displays.

Developer tools

Testing and quality tools

Vitest

Runs frontend unit tests. Command: npm test -- --run

jsdom

Lets tests act like they are running in a browser.

ESLint

Checks code style and catches common mistakes. Command: npm run lint

@vitejs/plugin-react

Connects React with Vite so JSX files work properly.

Useful commands

- npm install
- npm run dev
- npm test -- --run
- npm run lint
- npm run build

Backend connection

How frontend talks to backend

The frontend file frontend/src/api/aliClient.js sends a POST request to the Flask backend route:

```
POST http://localhost:5000/run
```

The frontend sends a small JSON body with demo settings. The backend returns generated code, final code, output, success status, stage, errors, and attempt count.

If backend is off

The frontend has a fallback demo response. This keeps the presentation from looking broken if Flask is not running, but the real project should use the backend whenever possible.

Design system

Visual style

- Pastel cream background so the site feels premium and soft.
- Black and dark brown text for readability.
- Maroon buttons and accents for a luxury minimalist look.
- Glassmorphism surfaces: soft borders, transparency, blur, and shadow.
- Animated network background to suggest APIs connecting.
- Large bold headline plus elegant italic line for personality.
- Logo mark in the navbar using the project logo with background removed.

Important rule

Do not randomly change colors, spacing, or fonts without checking the full page. The design depends on balance: cream background, maroon accents, dark readable text, and gentle glass layers.

Savneet's role

What Savneet should say

My role was to design and build the frontend experience for ALI. I created the website users interact with: the landing page, auth screens, upload interface, bridge progress timeline, generated-code display, result cards, and animations.

Simple explanation

The backend is the engine room. My frontend is the steering wheel, dashboard, and screen that lets users control the engine. A user should not need to understand code to use ALI.

What to demo

- Open the landing page and explain the project in one sentence.
- Click Sign Up or Log In to show the user flow.
- Open the tool page and load demo docs.
- Click Run ALI and show the timeline moving.
- Show the generated Python code and final result card.

How to maintain it

Where to change things

Change homepage text

Edit frontend/src/components/LandingPage.jsx.

Change colors or layout

Edit frontend/src/index.css.

Change backend URL

Edit apiBase inside frontend/src/api/aliClient.js.

Change pipeline steps

Edit frontend/src/data/pipeline.js.

Change logo

Replace frontend/src/assets/ali-logo-mark.png and keep the same filename.

Final summary

One-minute explanation

ALI is a website plus backend system that reads API docs and creates a working bridge between two tools. The frontend makes this process simple for users by giving them upload boxes, clear instructions, progress animation, generated code, and results.

Savneet's frontend in one line

I built the polished user interface that turns ALI from a backend tool into a product people can understand, use, and present.

Remember

- Frontend is what users see and click.
- Backend is what runs the logic.
- Sandbox is where generated code is tested safely.
- ALI is the smart layer that connects tools automatically.